



Introduction

In this lab session, you'll learn about format control codes with printf() function in c.

#include< stdio.h >

Both printf() and scanf() are predefined in the standard library **stdio.h** file, thus this header file should be always included if program uses printf() or scanf().

scanf() and printf () functions are I/O functions in C programming.

The function printf() is used for output.

Printf()

The printf function (stands for **print formatting**) is used to display information required by the user and also prints the values of the variables. For this, the printf function takes data values, converts them to a text stream using formatting functions specified in a control string and passes the resulting text stream to the standard output.

The syntax of **printf** function can be given as:

Syntax of printf (“control string”, variable list);

The function accepts two parameters **control string** and **variable list**.

The prototype of the control string can be as given below.

%[flags][width][.precision][length]specifier

Each control string must begin with a % character. The % character specifies how the next variable in the list of variables has to be printed.

After % sign follows.

Parameters

- **format** -- This is the string that contains the text to be written to stdout. It can optionally contain embedded format tags that are replaced by the values specified in subsequent additional arguments and formatted as requested. Format tags prototype is **%[flags][width][.precision][length]specifier**, which is explained below:

specifier	Output
c	Character.
d or i	Signed decimal integer
e	Scientific notation (mantissa/exponent) using e character
E	Scientific notation (mantissa/exponent) using E character
f	Decimal floating point
g	Use the shorter of %e or %f.
G	Use the shorter of %E or %f
o	Signed octal
s	String of characters
u	Unsigned decimal integer
x	Unsigned hexadecimal integer
X	Unsigned hexadecimal integer (capital letters)
p	Pointer address
n	Nothing printed.
%	Character.

flags	Description
-	Left-justify within the given field width; Right justification is the default (see width sub-specifier).
+	Forces to precede the result with a plus or minus sign (+ or -) even for positive numbers. By default, only negative numbers are preceded with a - sign.

(space)	If no sign is going to be written, a blank space is inserted before the value.
#	Used with o, x or X specifiers the value is preceeded with 0, 0x or 0X respectively for values different than zero. Used with e, E and f, it forces the written output to contain a decimal point even if no digits would follow. By default, if no digits follow, no decimal point is written. Used with g or G the result is the same as with e or E but trailing zeros are not removed.
o	Left-pads the number with zeroes (0) instead of spaces, where padding is specified (see width sub-specifier).

width	Description
(number)	Minimum number of characters to be printed. If the value to be printed is shorter than this number, the result is padded with blank spaces. The value is not truncated even if the result is larger.
*	The width is not specified in the format string, but as an additional integer value argument preceding the argument that has to be formatted.

.precision	Description
.number	For integer specifiers (d, i, o, u, x, X): precision specifies the minimum number of digits to be written. If the value to be written is shorter than this number, the result is padded with leading zeros. The value is not truncated even if the result is longer. A precision of 0 means that no character is written for the value 0. For e, E and f specifiers: this is the number of digits to be printed after the decimal point. For g and G specifiers: This is the maximum number of significant digits to be printed. For s: this is the maximum number of characters to be printed. By default all characters are printed until the ending null character is encountered. For c type: it has no effect. When no precision is specified, the default is 1. If the period is specified without an explicit value for precision, 0 is assumed.
.*	The precision is not specified in the format string, but as an additional integer value argument preceding the argument that has to be formatted.

length	Description
h	The argument is interpreted as a short int or unsigned short int (only applies to integer specifiers: i, d, o, u, x and X).
l	The argument is interpreted as a long int or unsigned long int for integer specifiers (i, d, o, u, x and X), and as a wide character or wide character string for specifiers c and s.
L	The argument is interpreted as a long double (only applies to floating point specifiers: e, E, f, g and G).

Flags

Specify output justification such as decimal point, numerical sign, trailing zeros or octal, decimal or hexadecimal prefixes.

Width

Specifies the minimum number of characters to be printed after being added with zeros or blank spaces, i.e., it specifies the minimum number of positions in the output.

Precision

Specifies the number of decimal places to be printed.

Specifier is used to define the type and the interpretation of the value of the corresponding argument.

Format Control Codes

In the printf () function the control string contains format commands that tells printf () how to display the remaining arguments and how many arguments are there. (See the following table)

CODE	MEANING	CODE	MEANING	CODE	MEANING	CODE	MEANING
%c	single character	%e	scientific notation	%f	floating point	%g	use %e or %f
%d	Decimal (base 10)	%o	Octal (base 8)	%p	pointer value	%s	string
%u	unsigned decimal	%x	hexa-decimal (base 16)				

Controlling integer width with printf()

The %3d specifier means a minimum width of three spaces, which, by default, will be right-justified:

printf("%3d", 0);	0
printf("%3d", 123456789);	123456789
printf("%3d", -10);	-10
printf("%3d", -123456789);	-123456789

Left-justifying printf() integer output

To left-justify integer output with printf(), just add a minus sign(-) after the % symbol, like this:

printf("%-3d", 0);	0
printf("%-3d", 123456789);	123456789
printf("%-3d", -10);	-10
printf("%-3d", -123456789);	-123456789

The printf() integer zero-fill option

To zero-fill your printf() integer output, just add a zero (0) after the % symbol, like this:

printf("%03d", 0);	000
printf("%03d", 1);	001
printf("%03d", 123456789);	123456789
printf("%03d", -10);	-10
printf("%03d", -123456789);	-123456789

Printf() integer formatting

As a summary, several different options are shown, including a minimum width specification, left-justified, zero-filled, and also a plus sign for positive numbers.

Description	Code	Result
At least five wide	<code>printf("%5d\n", 10);</code>	' 10'
At least five-wide, left-justified	<code>printf("%-5d\n", 10);</code>	'10 '
At least five-wide, zero filled	<code>printf("%05d\n", 10);</code>	'00010'
At least five-wide, with a plus sign	<code>printf("%+5d\n", 10);</code>	' +10'
five-wide, plus sign, left-justified	<code>printf("%-+5d\n", 10);</code>	'+10 '

Printf() formatting floating point numbers

Here are several examples showing how to format floating-point numbers with printf():

Description	Code	Result
Print one position after the decimal	<code>printf("%.1f", 10.3456);</code>	'10.3'
Two positions after the decimal	<code>printf("%.2f", 10.3456);</code>	'10.35'
Eight- wide, two positions after the decimal	<code>printf("%8.2f", 10.3456);</code>	' 10.35'
Eight- wide, four positions after the decimal	<code>printf("%8.4f", 10.3456);</code>	'10.3456'
Eight- wide, two positions after the decimal, zero filled	<code>printf("%08.2f", 10.3456);</code>	'00010.35'
Eight- wide, two positions after the decimal, left-justified	<code>printf("%-8.2f", 10.3456);</code>	'10.35 '
Printing much larger number with that same format	<code>printf("%-8.2f", 101234567.3456);</code>	'101234567.35'

Printf() string formatting

Here are several examples that show how to format string output with printf():

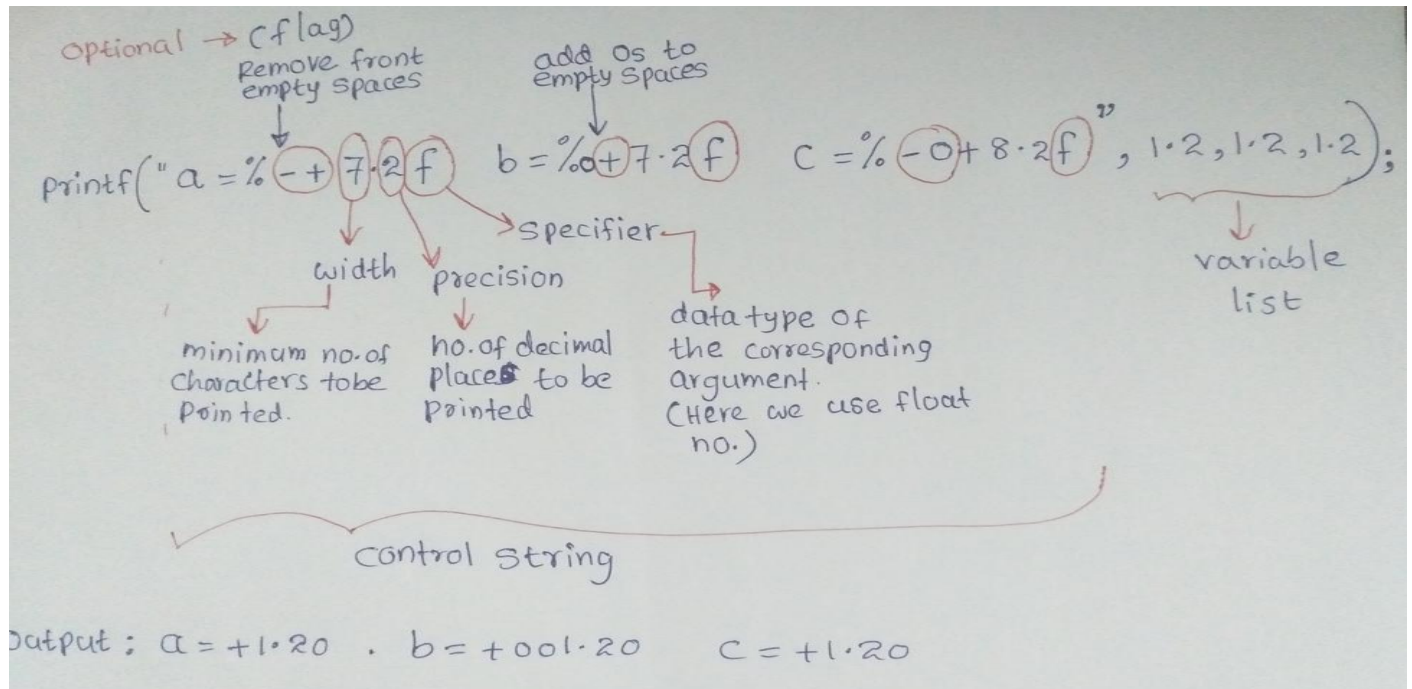
Description	Code	Result
A simple string	<code>printf("%s", "Hello");</code>	'Hello'
A string with a minimum length	<code>printf("%10s", "Hello");</code>	' Hello '
Minimum length, left-justified	<code>printf("%-10s", "Hello");</code>	'Hello '

`printf(“%E %e”, 321.65, 321.65);` **3.216500E+002 3.216500e+002**

`printf(“%G %g”, 321.65, 321.65);` **321.65 321.65**

e	Display the floating point number using specific notation with e 3.216500e+002
E	Like e, but with a capital E in the output 3.216500E+002
g	Use shorter of the two representations f or e
G	Like g, except uses the shorter of f or E
f	Display the floating point number using decimal representation is 321.65

See this example



```
printf("a=%-+7.2f b=%+7.2f c=%-0+8.2f \n",1.2,1.2,1.2);
```

```
a=+1.20 b=+001.20 c=+1.20
```

1. Write down the output of the following statements using one source file for all.

- | | |
|---|--|
| i. <code>printf("%d\n",2+3);</code> | vi <code>printf("%f\n",2.3+3.2);</code> |
| ii. <code>printf("\t%c\n",'A');</code> | vii <code>printf("\t%c\n",'AB');</code> |
| iii. <code>printf("\t%c\t%c\n",'A','B');</code> | viii <code>printf("%c\n",'ABCDE');</code> |
| iv. <code>printf("%s\n","ABCDE");</code> | ix <code>printf("%s\n","ABC","DEF");</code> |
| v. <code>printf("%s%s\n","ABC","DEF");</code> | x. <code>printf("%c%c%c%c%c%d\n",'2','+', '3','=',2+3);</code> |

2. Try out the following codes and get the output.

```
a.#include <stdio.h>
void main()
{
    printf( "<%d>\n", 336 );
    printf( "<%2d>\n", 336 );
    printf( "<%10d>\n", 336 );
    printf( "<%-10d>\n", 336 );
}
```

```

b.#include <stdio.h>
void main()
{
    printf( "<%f>\n", 1234.56 );
    printf( "<%e>\n", 1234.56 );
    printf( "<%4.2f>\n", 1234.56 );
    printf( "<%3.1f>\n", 1234.56 );
    printf( "<%10.3f>\n", 1234.56 );
    printf( "<%10.3e>\n", 1234.56 );
}

```

```

c.#include <stdio.h>
void main()
{
    printf( "<%2s>\n", "Paul must die!" );
    printf( "<%22s>\n", "Paul must die!" );
    printf( "<%22.5s>\n", "Paul must die!" );
    printf( "<%-22.5s>\n", "Paul must die!" );
}

```

Last line prints only 5 characters out of 22
--

```

d.
#include <stdio.h>

void main()
{
    printf("Printing floating-point numbers with\n");
    printf("floating-point conversion specifiers.\n");
    printf("Compare the output with source code\n\n");
    printf("1. %e\n", 1234567.89);
    printf("2. %e\n", +1234567.89);
    printf("3. %e\n", -1234567.89);
    printf("4. %E\n", 1234567.89);
    printf("5. %f\n", 1234567.89);
    printf("6. %g\n", 1234567.89);
    printf("7. %G\n", 1234567.89);
}

```

e.

```
#include <stdio.h>

int main()
{
    printf(" Printing integers right-justified.\n");
    printf("Compare the output with the source code\n");
    printf(".....\n\n");
    printf("%4d\n", 1);
    printf("%4d\n", 12);
    printf("%4d\n", 123);
    printf("%4d\n", 1234);
    printf("%4d\n\n", 12345);
    printf("%4d\n", -1);
    printf("%4d\n", -12);
    printf("%4d\n", -123);
    printf("%4d\n", -1234);
    printf("%4d\n", -12345);
    return 0;
}
```

f.

```
#include <stdio.h>

int main()
{
    int i = 873;
    float f = 123.94536;
    char s[ ] = "Happy Birthday";
    printf("Using precision while printing integers,\n");
    printf(" floating-point numbers, and strings.\n");
    printf("Compare the output with the source code\n");
    printf(".....\n\n");
    printf("Using precision for integers\n");
    printf("\t%.4d\t%.9d\n", i, i);
    printf("Using precision for floating-point numbers\n");
    printf("\t%.3f\t%.3e\t%.3g\n", f, f, f);
    printf("Using precision for strings\n");
    printf("\t%.11s\n", s);
    return 0;
}
```


g.

```
#include <stdio.h>

int main()
{
    printf("Right justifying and left justifying values.\n");
    printf(" Compare the output with the source code.\n");
    printf(".....\n\n");
    printf("%10s%10d%10c%10f\n\n", "hello", 7, 'a', 1.23);
    printf("%-10s%-10d%-10c%-10f\n", "hello", 7, 'a', 1.23);
    return 0;
}
```

h.

```
#include <stdio.h>

int main()
{
    printf("Printing numbers with and without the + flag.\n");
    printf(" Compare the output with the source code.\n");
    printf(".....\n\n");
    printf("%d\n%d\n", 786, -786);
    printf("%+d\n%+d\n", 786, -786);
    return 0;
}
```

i.

```
#include <stdio.h>

int main()
{
    printf("Printing a space before signed values\n");
    printf(" not preceded by + or -\n");
    printf(".....\n\n");
    printf("% d\n% d\n", 877, -877);
    return 0;
}
```

j.

```
#include <stdio.h >
int main()
{
    int c = 1427; float p = 1427.0;
    printf("o, x, X, and any floating-point specifiers\n");
    printf("Compare the output with the source code\n");
    printf(".....\n\n");
    printf("%#o\n", c);
    printf("%#x\n", c);
    printf("%#X\n", c);
    printf("\n%#g\n", p);
    printf("%#G\n", p);
    return 0;
}
```

k.

```
#include <stdio.h>
int main()
{
    printf("Printing with the 0 (zero) flag fills in leading zeros\n");
    printf("    Compare the output with the source code\n");
    printf(".....\n\n");
    printf("%+09d\n", 762);
    printf("%09d", 762);
    printf("\n");
    return 0;
}
```